

```

# =====
# ARTIFICIAL NEURAL NETWORK USING BACKPROPAGATION
# Dataset: XOR
# Run directly in Google Colab
# =====

import numpy as np

# -----
# 1. Create XOR Dataset
# -----

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)

print("Input Data:")
print(X)

print("\nTarget Output:")
print(y)

# -----
# 2. Define Activation Functions
# -----

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# -----
# 3. Initialize Neural Network Parameters
# -----

np.random.seed(42)

# Network architecture:
# Input layer : 2 neurons
# Hidden layer : 4 neurons
# Output layer : 1 neuron

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

# Weights
W1 = np.random.uniform(-1, 1, (input_neurons, hidden_neurons))
W2 = np.random.uniform(-1, 1, (hidden_neurons, output_neurons))

# Biases
b1 = np.zeros((1, hidden_neurons))
b2 = np.zeros((1, output_neurons))

# Learning rate
learning_rate = 0.5

# Number of training iterations
epochs = 10000

```

```

# -----
# 4. Backpropagation Training
# -----

for epoch in range(epochs):

    # -----
    # Forward Propagation
    # -----

    # Hidden layer
    z1 = np.dot(X, W1) + b1
    a1 = sigmoid(z1)

    # Output layer
    z2 = np.dot(a1, W2) + b2
    a2 = sigmoid(z2)

    # -----
    # Calculate Error
    # -----

    error = y - a2

    # Mean Squared Error
    mse = np.mean(error ** 2)

    # -----
    # Backpropagation
    # -----

    # Output layer gradient
    d_output = error * sigmoid_derivative(a2)

    # Hidden layer gradient
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(a1)

    # -----
    # Update Weights and Biases
    # -----

    W2 += np.dot(a1.T, d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

    W1 += np.dot(X.T, d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    # Display error periodically
    if epoch % 1000 == 0:
        print(f"Epoch {epoch:5d} | MSE = {mse:.6f}")

# -----
# 5. Test the Trained Neural Network
# -----

z1 = np.dot(X, W1) + b1
a1 = sigmoid(z1)

z2 = np.dot(a1, W2) + b2
predictions = sigmoid(z2)

# Convert probabilities to 0 or 1
predicted_classes = (predictions >= 0.5).astype(int)

# -----
# 6. Display Results
# -----

print("\n" + "=" * 50)
print("FINAL RESULTS")
print("=" * 50)

print("\nInput\t\tTarget\t\tPredicted")
print("-" * 50)

for i in range(len(X)):

```

```

.....print(
.....    f"{X[i].astype(int)}\t\t"
.....    f"{y[i][0]:.0f}\t\t"
.....    f"{predictions[i][0]:.4f} -> "
.....    f"{predicted_classes[i][0]}"
.....)

# -----
# 7. Calculate Accuracy
# -----

accuracy = np.mean(predicted_classes == y) * 100

print("\nAccuracy:", accuracy, "%")

# -----
# 8. Display Learned Weights
# -----

print("\nFinal Hidden Layer Weights:")
print(W1)

print("\nFinal Output Layer Weights:")
print(W2)

print("\nFinal Hidden Layer Bias:")
print(b1)

print("\nFinal Output Layer Bias:")
print(b2)

```

Input Data:

```

[[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]]

```

Target Output:

```

[[0.]
 [1.]
 [1.]
 [0.]]

```

```

Epoch    0 | MSE = 0.262916
Epoch 1000 | MSE = 0.005853
Epoch 2000 | MSE = 0.001787
Epoch 3000 | MSE = 0.001019
Epoch 4000 | MSE = 0.000705
Epoch 5000 | MSE = 0.000536
Epoch 6000 | MSE = 0.000431
Epoch 7000 | MSE = 0.000360
Epoch 8000 | MSE = 0.000309
Epoch 9000 | MSE = 0.000270

```

=====

FINAL RESULTS

=====

Input	Target	Predicted
[0 0]	0	0.0095 -> 0
[0 1]	1	0.9839 -> 1
[1 0]	1	0.9849 -> 1
[1 1]	0	0.0195 -> 0

Accuracy: 100.0 %

Final Hidden Layer Weights:

```

[[-0.40202049  4.60779894  6.37090183  3.82832237]
 [-1.20298037 -6.26244632 -4.77443726  3.84663039]]

```

Final Output Layer Weights:

```

[[ 1.5059499 ]
 [ 8.92519656]
 [-9.08595643]
 [ 4.44147084]]

```

Final Hidden Layer Bias:

```

[[ 0.48214239 -2.05767116  2.09458143 -0.59838294]]

```

```
Final Output Layer Bias:  
[[-0.07516984]]
```